哈爾濱工業大學 HARBIN INSTITUTE OF TECHNOLOGY （深圳）

实验报告

开课学期：2025 秋季

课程名称：计算思维与实践

实验名称：实验十一：音乐播放管理器

实验性质：设计型

实验学时：2 地点：T2617

学生学号：2025311510

学生姓名：赵逸飞

评阅教师：

报告成绩：

实验与创新实践教育中心制

2025 年 12 月

一、实验目的

1. 掌握单向链表的基本概念及常见操作；
2. 理解链表在动态数据管理中的应用场景；
3. 掌握文件读写操作，实现数据的持久化存储；
4. 提升程序设计与综合调试能力。

二、实验内容

设计并实现一个简单的音乐播放器歌单管理系统，使用单向链表存储歌曲信息，实现以下功能：

1. 从文件中读取歌曲信息，并建立歌单链表；
2. 支持用户添加新的歌曲信息；
3. 支持用户删除指定歌曲；
4. 支持按歌曲名查找并“播放”歌曲；
5. 显示当前歌单列表；
6. 将更新后的歌单信息写回文件。

三、系统设计（40 分）

3.1 数据结构设计

//结构体定义

```
typedef struct song {
    int id;                  // 歌曲 id
    char title[100];         // 歌曲标题（最多 99 个字符）
    char artist[50];         // 歌手名称（最多 49 个字符）
    char filepath[300]; // 歌曲文件路径（最多 299 个字符）
    struct song* next;   // 指向下一个结点的指针
} Song;
//
typedef struct playlistmanager {
    Song* head;          // 链表头结点
    Song* tail;          // 链表尾结点
    Song* current;   // 当前播放结点
} PlaylistManager;
```

单向链表，有头节点，头结点 → 第一个歌曲结点 → 第二个歌曲结点 → … → 尾结点（next 为 NULL）；

3.2 程序总体结构设计

说明主函数 main() 的功能，各子函数之间的调用关系，以及程序的整体运行流程。请绘制程序流程图或模块调用关系图。

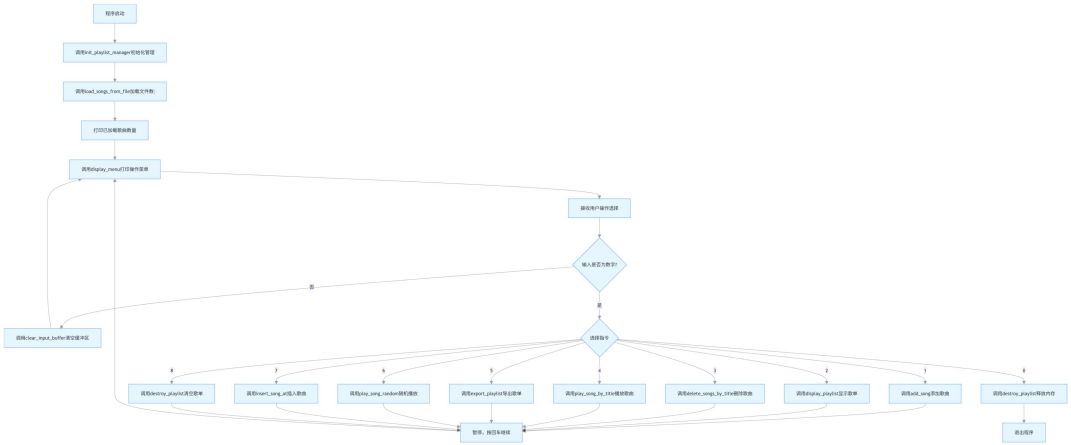
main()函数是程序的入口，核心功能包括：

1.初始化歌单管理器，从文件加载歌曲数据并创建链表；

2.打印操作菜单，接收用户输入的操作指令；

3.通过 switch 语句分发指令，调用对应功能函数；

4.循环执行直到用户选择退出程序。



四、函数设计（40 分）

4.1 函数接口定义

列出程序中主要函数的函数名、参数说明、返回值及其功能描述。

函数名	参数说明	返回值	功能说明
init_playlist_manager	PlaylistManager* manager	void	初始化管理器，所有指针置 NULL，计数置 0
load_songs_from_file	manager：管理器指针；filename：读取文件路径	int	从文件读取歌曲数据构建链表，返回加载数量
add_song	manager：管理器指针；title/artist/filepath：歌曲信息	void	向链表末尾添加歌曲，自动生成 ID
display_playlist	PlaylistManager* manager：歌单管理器指针	void	遍历链表，显示所有歌曲信息
delete_songs_by_title	manager：管理器指针；title：要删除的歌曲标题	int	按标题删除歌曲，返回删除数量
play_song_by_title	manager：管理器指针；title：要播放的歌曲标题	int	按标题查找并播放歌曲，成功返回 1，失败返回 0
export_playlist	manager：管理器指针；filename：	int	将链表数据写入指定文件，成功

函数名	参数说明	返回值	功能说明
	导出文件路径		返回 1，失败返回 0
play_song_random	PlaylistManager* manager：歌单管理器指针	int	随机选择一首歌曲播放，成功返回 1，失败返回 0
insert_song_at	manager：管理器指针；position：插入位置；title/artist/filepath：歌曲信息	int	在指定位置插入歌曲，成功返回 1，失败返回 0
destroy_playlist	PlaylistManager* manager：歌单管理器指针	void	清空链表所有结点，释放内存并重置管理器

4.2 主要函数功能说明

分别说明以下函数的设计思路和实现逻辑：

1. 从文件读取歌曲信息并建立链表的函数；
   打开指定文件，按行读取歌曲信息（id, 标题，歌手，路径）；
   每读取一条信息，创建新结点并初始化数据；
   将新结点连接到链表末尾，更新 tail 指针和 song_count；
   读取完成后关闭文件，返回成功加载的歌曲数量；
   包含文件打开失败、内存分配失败的异常处理。

2. 添加歌曲信息的函数；
   为新歌曲分配结点内存，检验内存分配是否成功；
   自动生成 id（空链表时为 1，否则为尾结点 id+1）；
   使用 strncpy 安全拷贝标题、歌手、路径，避免缓冲区溢出；
   将新结点挂载到链表末尾，更新 tail 指针和 song_count；
   打印添加成功提示，无文件写入（导出功能统一处理持久化）。

3. 删除指定歌曲的函数；
   采用“前驱结点遍历法”：遍历指针 cur 指向当前结点，prev 指向前驱结点；
   对比 cur 的标题与目标标题，匹配时删除该结点；
   若删除的是头结点，更新 head 指针；若删除的是尾结点，更新 tail 指针；
   释放删除结点的内存，更新 song_count，统计并返回删除数量；
   支持删除多个同名歌曲

4. 按歌曲名查找并播放歌曲的函数；
   遍历链表，对比每个结点的标题与目标标题；
   匹配成功时调用 play_audio 播放音频文件，打印播放提示；
   匹配失败时打印“未找到歌曲”提示，返回对应状态码。

5. 显示歌单列表的函数；
   先判断链表是否为空，为空则打印提示；
   非空则遍历链表，逐行打印每个结点的 ID、标题、歌手、路径；
   打印开头显示歌曲总数，提升用户体验。

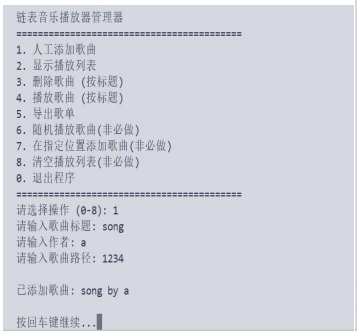
6. 将链表数据写回文件的函数。
   以覆盖模式（w）打开指定文件，校验文件是否打开成功；
   遍历链表，将每个结点的信息按“ID, 标题，歌手，路径”格式写入文件；
   写入完成后关闭文件，打印导出成功提示，返回对应状态码。

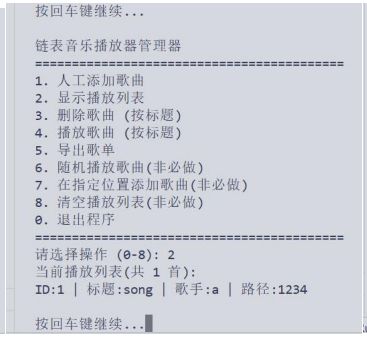
五、实验结果与测试（20 分）

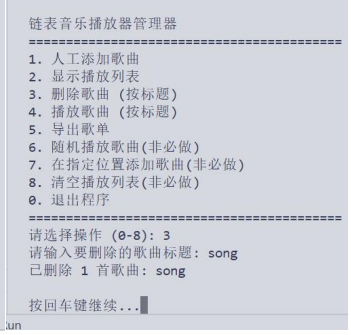
5.1 测试用例设计

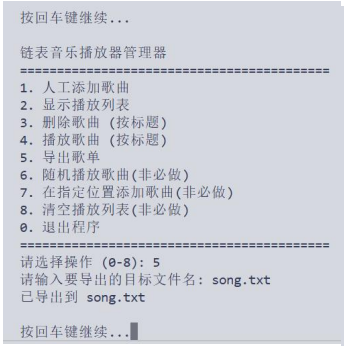
正常输入	添加并显示歌曲	1. 选择 1，输入标题 “晴天”、歌手 “周杰伦”、路径 “D:\music\ 晴天.mp3”；2. 选择 2 显示歌单	歌单中显示新增歌曲，ID 为 1（空链表）或尾结点 ID+1
正常输入	删除指定歌曲	1. 添加歌曲 “晴天”；2. 选择 3，输入 “晴天”；3. 选择 2 显示	歌单中无 “晴天”，提示 “已删除 1 首歌曲：晴天”
正常输入	播放指定歌曲	1. 添加有效路径的歌曲 “晴天”；2. 选择 4，输入 “晴天”	提示 “正在播放”，弹出系统播放器播放音频
正常输入	导出歌单	1. 添加 2 首歌曲；2. 选择 5，输入导出路径 “my_playlist.txt”	生成 my_playlist.txt，内容与歌单一致
边界情况	空歌单操作	1. 删除所有歌曲；2. 选择 2/3/4/6	提示 “播放列表为空”，程序不崩溃
边界情况	仅一个结点的歌单	1. 添加 1 首歌曲；2. 删除该歌曲；3. 显示歌单	歌单为空，删除成功提示
边界情况	插入到末尾位置	1. 添加 2 首歌曲；2. 选择 7，插入位置 2；3. 显示歌单	新歌曲出现在歌单最后，song_count+1
异常情况	播放不存在的歌曲	1. 选择 4，输入 “七里香”	提示 “未找到歌曲：七里香”，程序正常运行
异常情况	删除不存在的歌曲	1. 选择 3，输入 “七里香”	提示 “未找到歌曲：七里香”，程序正常运行
异常情况	插入无效位置（负数 / 超出范围）	1. 选择 7，输入位置 -1 或 100；2. 输入歌曲信息	提示 “插入位置无效”，未添加歌曲
异常情况	播放无效路径的歌曲	1. 添加路径错误的歌曲；2. 选择 4 播放	提示 “无法打开文件”，程序不崩溃
异常情况	非数字输入操作指令	1. 菜单选择时输入 “a”	提示 “无效输入，请输入数字”，返回菜单

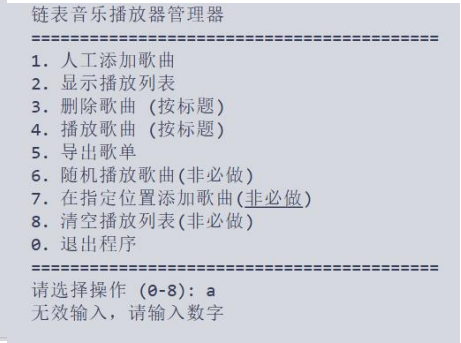
5.2 测试结果展示











六、问题与解决方法

问题 ：输入过长的歌曲标题时，程序出现乱码甚至崩溃；

解决方法：直接使用 strcpy 拷贝字符串，未限制长度，超出字符数组范围导致缓冲区溢出，改用 strncpy，限制拷贝长度为数组长度 - 1，并手动在末尾添加 '\0'，确保字符串终止符有效

问题：有时候写完代码再回看会看不懂自己在写什么

解决方法：利用注释，简洁地标注一段代码地作用

问题：关于边界和异常情况考虑不够全面

解决方法：尽量考虑全面，会参考之前所学的内容或做过的项目

七、课程总结与建议

1.本次实验通过实现音乐播放管理器，深入掌握了单向链表的核心操作（创建、插入、删除、遍历）

2.认识到异常处理和边界条件校验的重要性，如空链表、无效输入、内存分配失败、文件路径错误等场景的处理，是保证程序健壮性的关键

3.理解了模块化程序设计的优势，将功能拆分为独立函数，提升了代码的可读性、可维护性和复用性

4.希望可以有更多的练习，实验有点难。项目有点大